most REST APIs, Ninja is sufficient. Ninja is also a younger library, and features are getting added frequently; it may handle your need soon.

I've mentioned several times in this chapter the importance of a REST API's documentation, especially considering how loosely defined the protocol is. While both Ninja and the DRF come with web-based interfaces that can be used as docs, the developers consuming your API would appreciate actual documentation. Since API endpoints get exposed through views and classes in Ninja and DRF, the usual Python tools are sufficient. My personal preference is Sphinx (https://www.sphinx-doc.org/), but any Python documentation tool will work.

There are non-REST alternatives to providing an API as well. A popular mechanism is GraphQL. A GraphQL interface has a single endpoint that takes a JSON-like document that specifies a query. There are no HTTP methods to "verb" an object, and you don't need to specify multiple URL endpoints.

To build a GraphQL API interface with Django, you use a third-party library called Graphene (https://docs.graphene-python.org). Like with Ninja or the DRF, you specify a series of schemas that indicate what data can be returned from your interface and how it maps to the ORM. Since GraphQL doesn't use HTTP methods, you also need to define query classes that specify those actions that can be performed. Different types of query classes map to CRUD operations. This is essentially similar to REST endpoints, but it is built into the query language instead.

Graphene schemas and query classes get registered against a single `GraphQLView` object, which, itself, is registered as a view. This view is your API's endpoint. I have written systems that interact with upstream servers using GraphQL, but I've never built an API based upon it. The argument for GraphQL is flexibility on the client side. By specifying a schema, you get filtering and more complex queries for free. For example, you can ask for an object and limit what data comes back, minimizing your bandwidth usage. It is akin to an SQL call, where you can limit the response to only the columns you're interested in. There are two main arguments against GraphQL. First, the flexibility comes with the cost of the developer using the API having to write the queries to consume the data. Contrast this with REST, where they simply hit a URL. The second argument against is a provisioning one: due to the flexibility, the load on your server gets determined by the complexity of the query the downstream developer has written. This makes it much harder to predict load cost.

If you're writing your first API, I'd suggest sticking with Ninja. If you're already comfortable with GraphQL from other projects or frameworks, then Graphene could be the answer you're looking for.

When weighing which API framework to use, consider that Ninja has a few other features not covered in this chapter. For example, it supports the following:

- Endpoints implementing multiple HTTP methods
- HTTP methods not covered by the decorators
- Pagination
- Dynamically creating schemas